

MACHINE LEARNING IN FINANCE

MODULE 7 | LESSON 2

HOUSE PREDICTION AND HYPERPARAMETER TUNING

Reading Time	35 minutes
Prior Knowledge	Grid Search, Random Search
Keywords	Grid search

In the last lesson, we introduced hyperparameter tuning in machine learning and the merits and demerits of the various algorithms used. In this lesson, we will apply the skills learned to a house prediction exercise.

1. Introduction

In this lesson we will use the housing data from the [Geo Data and Lab](#) website to predict the housing prices using Random Forest Regression and later on fine-tune its hyperparameters so as to improve the model performance.

We begin by first loading the relevant packages below.

```
In [ ]: # data manipulation and plotting
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# from Scikit-Learn
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
```

We then load the downloaded file to our working space and read it.

```
In [ ]: df = pd.read_csv("kc_house_data.csv")
```

```
In [ ]: df.head()
```

Since the data is already processed, we will lightly touch on the exploratory data analysis phase and concentrate mostly on the modeling phase.

Below we can see the size of the data and examine missing values.

2. Exploratory Data Analysis

```
In [ ]: print(f"The dataset contains {df.shape[0]} samples and " f"{df.shape[1]} feature
```

```
In [ ]: df.isnull().sum()
```

We then drop the id and date column from our data as they will not be useful for our prediction exercise. We also separate the target variable from the predictor variables.

```
In [ ]: X = df.drop(["id", "price", "date"], axis=1)
y = df["price"]
```

The next step is to pick out categorical variables from numerical variables using column selector from `sklearn` 's `compose`.

```
In [ ]: from sklearn.compose import make_column_selector as selector

categorical_columns_selector = selector(dtype_include=object)
categorical_columns = categorical_columns_selector(X)
categorical_columns
```

```
In [ ]: # now let's identify the numerical variables

num_vars = [var for var in X.columns if var not in categorical_columns]

# number of numerical variables
print(len(num_vars))
print(num_vars)
```

We note that some categorical variables are numerical and we can put them into their correct list.

```
In [ ]: # let's make a list of discrete variables
categorical_vars = [var for var in num_vars if len(X[var].unique()) < 20]

print("Number of categorical variables: ", len(categorical_vars))
print(categorical_vars)
```

```
In [ ]: # let's visualize the categorical variables

X[categorical_vars].head()
```

Therefore, the remaining numerical variables will now be classified as continuous variables.

```
In [ ]: # make list of continuous variables
cont_vars = [var for var in num_vars if var not in categorical_vars]

print("Number of continuous variables: ", len(cont_vars))
print(cont_vars)
```

```
In [ ]: # Let's visualize the continuous variables
```

```
X[cont_vars].head()
```

We now visualize the variables' distribution using histograms as shown below.

```
In [ ]: # Lets plot histograms for all continuous variables
```

```
X[cont_vars].hist(bins=30, figsize=(15, 15))
plt.suptitle("Fig. 1: Histogram Plot", fontweight="bold", horizontalalignment="r")
plt.show()
```

We can now standardize the continuous variables and encode the discrete variables to eliminate possible bias towards larger values.

3. Modeling Pipeline

```
In [ ]: from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
categorical_preprocessor = OneHotEncoder(handle_unknown="ignore")
numerical_preprocessor = StandardScaler()
```

```
In [ ]: from sklearn.compose import ColumnTransformer
```

```
preprocessor = ColumnTransformer(
    [
        ("one-hot-encoder", categorical_preprocessor, categorical_vars),
        ("standard_scaler", numerical_preprocessor, cont_vars),
    ]
)
```

We now create a pipeline of our regression model as shown below.

```
In [ ]: model = Pipeline(
    [
        ("preprocessor", preprocessor),
        ("regressor", RandomForestRegressor(random_state=42)),
    ]
)
```

```
In [ ]: from sklearn import set_config
```

```
set_config(display="diagram")
model
```

The data is now split into training data and testing data.

```
In [ ]: # Let's separate into train and test set
# Remember to set the seed (random_state for this `sklearn` function)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, # predictive variables
    y, # target
    test_size=0.3, # portion of dataset to allocate to test set
```

```

    random_state=0, # we are setting the seed here
)

X_train.shape, X_test.shape

```

We can note below that the target variable is skewed to the left and our model will have a tendency to predict values to the most common value. Therefore, we transform it to make it more normal.

```
In [ ]: # histogram to evaluate target distribution
```

```

df["price"].hist(bins=50, density=True)
plt.ylabel("Number of houses")
plt.xlabel("Sale price")
plt.suptitle(
    "Fig. 2: Target Variable Distribution",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()

```

After log transforming the target variable, it now looks more normally distributed and will therefore use the log transform variable as our target.

```
In [ ]: # Let's transform the target using the Logarithm
```

```

np.log(df["price"]).hist(bins=50, density=True)
plt.ylabel("Number of houses")
plt.xlabel("Log of Sale Price")
plt.suptitle(
    "Fig. 3: Log Transformed Target", fontweight="bold", horizontalalignment="ri
)
plt.show()

```

```
In [ ]: y_train = np.log(y_train)
        y_test = np.log(y_test)
```

```
In [ ]: X_train.head()
```

We now train the model using the pipeline we formulated above.

```
In [ ]: _ = model.fit(X_train, y_train)
```

```
In [ ]: X_test.head()
```

We then use the model to make predictions about unseen test data and evaluate its performance.

```

In [ ]: target_predicted = model.predict(X_test)

print(
    f"Mean squared error on the testing set: "
    f"{mean_squared_error(y_test, target_predicted):.3f}"
)

```

4. Regression Metrics

There are various metrics we can use to evaluate the performance of a regression model.

```
In [ ]: model.score(X_test, y_test)
```

The model performed really well, and we will now want to see if we can improve this performance using hyperparameter tuning.

```
In [ ]: from sklearn.dummy import DummyRegressor

dummy_regressor = DummyRegressor(strategy="mean")
dummy_regressor.fit(X_train, y_train)
print(
    f"R2 score for a regressor predicting the mean:"
    f"{dummy_regressor.score(X_test, y_test):.3f}"
)
```

```
In [ ]: from sklearn.metrics import mean_absolute_error

print(f"Mean absolute error: " f"{mean_absolute_error(y_test, target_predicted):
```

5. Hyperparameter Tuning

5.1 Random Search

We will need to define the hyperparameter grid to be able to use random search.

```
In [ ]: from pprint import pprint

# Look at parameters used by our current forest
print("Parameters currently in use:\n")
pprint(model.get_params())
```

Random search has the advantage of being fast as it does not use all the values in the hyperparameter space but rather randomly selects variables.

```
In [ ]: from sklearn.model_selection import RandomizedSearchCV

# Number of trees in random forest
n_estimators = [int(x) for x in np.linspace(start=10, stop=90, num=10)]
# Number of features to consider at every split
max_features = ["auto", "sqrt"]
# Maximum number of levels in tree
max_depth = [int(x) for x in np.linspace(10, 50, num=21)]
max_depth.append(None)
# Minimum number of samples required to split a node
min_samples_split = [2, 5, 10]
# Minimum number of samples required at each leaf node
min_samples_leaf = [1, 2, 4]
# Method of selecting samples for training each tree
bootstrap = [True, False]
# Create the random grid
```

```

random_grid = {
    "regressor__n_estimators": n_estimators,
    "regressor__max_features": max_features,
    "regressor__max_depth": max_depth,
    "regressor__min_samples_split": min_samples_split,
    "regressor__min_samples_leaf": min_samples_leaf,
    "regressor__bootstrap": bootstrap,
}
pprint(random_grid)

```

We now create a model with all the parameters specified above.

```

In [ ]: model_random_search = RandomizedSearchCV(
        model,
        param_distributions=random_grid,
        n_iter=10,
        cv=5,
        verbose=1,
    )
model_random_search.fit(X_train, y_train)

```

We can now visualize the best hyperparameters that we will fit back to our model.

```

In [ ]: model_random_search.best_params_

```

By using this hyperparameter values, we now evaluate our algorithm's performance.

```

In [ ]: def evaluate(model, test_features, test_labels):
        predictions = model.predict(test_features)
        errors = abs(predictions - test_labels)
        mape = 100 * np.mean(errors / test_labels)
        accuracy = 100 - mape
        print("Model Performance")
        print("Average Error: {:.4f} dollars.".format(np.mean(errors)))
        print("Accuracy = {:.2f}%.".format(accuracy))

        return accuracy

base_model = RandomForestRegressor(n_estimators=10, random_state=42)
base_model.fit(X_train, y_train)
base_accuracy = evaluate(base_model, X_test, y_test)

```

```

In [ ]: best_random = model_random_search.best_estimator_
random_accuracy = evaluate(best_random, X_test, y_test)

```

Comparing the performance of our model without and with hyperparameter tuning show us a marginal increase in accuracy.

```

In [ ]: print(
        "Improvement of {:.2f}%.".format(
            100 * (random_accuracy - base_accuracy) / base_accuracy
        )
    )

```

5.2 Grid Search

We now provide a framework for performing grid search on our model hyperparameters. Note that we restricted the hyperparameters to a smaller space since the grid search algorithm is very heavy. We therefore do not expect a better performance. Students are allowed to experiment with bigger spaces and see how the model performs.

```
In [ ]: from sklearn.model_selection import GridSearchCV

# Create the parameter grid based on the results of random search
param_grid = {
    "regressor__bootstrap": [True],
    "regressor__max_depth": [int(x) for x in np.linspace(10, 20, num=6)],
    "regressor__max_features": [2, 3],
    "regressor__min_samples_leaf": [3, 4, 5],
    "regressor__min_samples_split": [8, 10, 12],
    "regressor__n_estimators": [
        int(x) for x in np.linspace(start=70, stop=100, num=11)
    ],
}
# Create a based model
rf = RandomForestRegressor()
# Instantiate the grid search model
grid_search = GridSearchCV(
    estimator=model, param_grid=param_grid, cv=3, n_jobs=-1, verbose=2
)
```

```
In [ ]: # Fit the grid search to the data
grid_search.fit(X_train, y_train)
grid_search.best_params_
```

```
In [ ]: best_grid = grid_search.best_estimator_
grid_accuracy = evaluate(best_grid, X_test, y_test)
```

We also compare the performance of the grid search algorithm with that of the original random forest algorithm.

```
In [ ]: print(
    "Improvement of {:.2f}%".format(
        100 * (grid_accuracy - base_accuracy) / base_accuracy
    )
)
```

Conclusion

In this lesson, we have seen how to apply hyperparameter tuning to a regression problem. In the next lesson, we will study objective functions and other techniques used to evaluate machine learning models.

References

1. Breiman, Leo. "Random Forests." *Machine Learning*, vol. 45, no. 1, 2001, pp 5-32.

2. GeoDa Data and Lab. "2014-15 Home Sales in King County, WA."
<https://geodacenter.github.io/data-and-lab/KingCounty-HouseSales2015/>
3. Ramadhan, Muhammad Murtadha, et al. "Parameter Tuning in Random Forest based on Grid Search Method for Gender Classification based on Voice Frequency."
DEStech Transactions on Computer Science and Engineering, vol. 10, 2017.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use.
Redistribution or publication of this material is strictly prohibited.